

EST-1141

Lenguajes de Programación

Juan Zamora O.

Programación Lógica con PROLOG



PONTIFICIA
UNIVERSIDAD
CATÓLICA DE
VALPARAÍSO

Estructura

- 1 Agenda
- 2 Cálculo de predicados de primer orden
- 3 Uso de la lógica en un LP
- 4 Aspectos básicos de Prolog

Agenda

- Cálculo de predicados de primer orden
- Uso de la lógica en un lenguaje de programación
- Aspectos básicos de Prolog

Cálculo de predicados de primer orden

- Extensión de la lógica proposicional
- Permite expresar información sobre **objetos** y **relaciones** entre ellos
- Comprende:
 - Símbolos para representar individuos: constantes, variables, funciones y predicados
 - Conectores
 - Cuantificadores
 - Símbolos de puntuación
 - Reglas de inferencia para derivar nuevas declaraciones a partir de un conjunto dado de declaraciones
- Los **predicados** son funciones que solo generan un valor `True` o `False`

Operadores lógicos y su orden de precedencia

- 1 $\neg$
- 2 $\wedge$
- 3 $\vee$
- 4 $\rightarrow, \leftrightarrow$
- 5 $\forall, \exists$

Así, la expresión

$$((\neg b) \wedge c) \rightarrow a$$

puede simplificarse a

$$\neg b \wedge c \rightarrow a$$

Equivalencias lógicas

$$\neg\neg f \equiv f$$

$$f \rightarrow g \equiv \neg f \vee g$$

$$f \leftrightarrow g \equiv (f \rightarrow g) \wedge (g \rightarrow f)$$

$$\neg(f \vee g) \equiv \neg f \wedge \neg g$$

$$\neg(f \wedge g) \equiv \neg f \vee \neg g$$

$$\neg\forall x. f(x) \equiv \exists x. \neg f(x)$$

$$\neg\exists x. f(x) \equiv \forall x. \neg f(x)$$

Equivalencias con cuantificadores

$$\forall x. (f(x) \wedge g(x)) \equiv (\forall x. f(x)) \wedge (\forall x. g(x))$$

$$\forall x. (f(x) \vee g(x)) \not\equiv (\forall x. f(x)) \vee (\forall x. g(x))$$

$$\exists x. (f(x) \vee g(x)) \equiv (\exists x. f(x)) \vee (\exists x. g(x))$$

$$\exists x. (f(x) \wedge g(x)) \not\equiv (\exists x. f(x)) \wedge (\exists x. g(x))$$

Reglas de inferencia

$$\frac{f \quad f \rightarrow g}{g}$$

$$\frac{\forall x. f(x)}{f(t)}$$

$$\frac{f(t)}{\exists x. f(x)}$$

$$\frac{(a \rightarrow b) \quad (b \rightarrow c)}{a \rightarrow c}$$

$$\frac{f \quad g}{f \wedge g}$$

Ejemplos de sentencias lógicas

- ① Un caballo es un mamífero
- ② Un humano es un mamífero
- ③ Los mamíferos tienen 4 piernas y no tienen brazos, o bien tienen dos piernas y dos brazos
- ④ Un caballo no tiene brazos
- ⑤ Un humano tiene brazos
- ⑥ Un humano no tiene piernas

Traducción a lógica de primer orden

Una traducción posible de estas declaraciones es:

```
mamifero(caballo).
```

```
mamifero(humano).
```

```
% Para todo X:
```

```
%   mamifero(X) -> (piernas(X, 4) and brazos(X, 0))
```

```
%               or (piernas(X, 2) and brazos(X, 2)).
```

```
brazos(caballo, 0).
```

```
not brazos(humano, 0).
```

```
piernas(humano, 0).
```

Análisis del ejemplo

- X es la única **variable**
- 0, 2 y 4 son **constantes**, además de los nombres caballo y humano
- brazos, piernas y mamifero son **predicados**

A partir de las primeras 5 declaraciones es posible derivar los siguientes teoremas:

piernas(caballo, 4).

piernas(humano, 2).

brazos(humano, 2).

La esencia de la programación lógica

Una colección de declaraciones asumidas como **axiomas**, a partir de las cuales se derivan hechos mediante la aplicación **automática** de reglas de inferencia.

Lenguaje de programación lógica

- Es un sistema notacional que permite escribir declaraciones lógicas
- Tiene además un conjunto de algoritmos que implementan reglas de inferencia
- Las declaraciones lógicas consideradas como axiomas conforman el **programa lógico**
- Las declaraciones a derivar corresponden a la **entrada** que da inicio al cómputo
 - Estas declaraciones se denominan **consultas (queries)**

Para el ejemplo anterior, podríamos pensar en la consulta:

¿Existe algún Y tal que Y corresponde al número de piernas de un humano?

Hechos y reglas

- La idea de la programación lógica es usar un computador para derivar conclusiones a partir de descripciones declarativas
- Esto se logra introduciendo sentencias declarativas que describen hechos positivos y reglas
 - **Hecho:** indica que una relación se cumple entre los individuos
 - **Regla:** indica que una relación se cumple entre los individuos siempre que otras relaciones se cumplan

Ejemplo: relaciones familiares

- i. Juan es hijo de Pablo
- ii. Ana es hija de Juan
- iii. Pablo es hijo de Marco
- iv. Alicia es hija de Pablo
- v. El nieto de una persona es el hijo de un hijo de esta persona

Formalización en dos pasos

Paso 1 — fórmulas atómicas que describen hechos:

hijo(juan, pablo).

hijo(ana, juan).

hijo(pablo, marco).

hijo(alicia, pablo).

Paso 2 — reglas sobre los predicados usados en los hechos:

Para todo X e Y, nieto(X, Y) si existe un Z tal que hijo(X, Z) y hijo(Z, Y).

$$\forall X \forall Y (\text{nieto}(X, Y) \leftarrow \exists Z (\text{hijo}(X, Z) \wedge \text{hijo}(Z, Y)))$$

Cláusulas definidas

Centraremos nuestra atención en fórmulas (**cláusulas de Horn**) de la forma:

$$A_0 \leftarrow A_1 \wedge \dots \wedge A_n, \quad n \geq 0$$

- Los $A_0, \dots, A_n$ son fórmulas atómicas
- Todas las variables de la fórmula están cuantificadas universalmente sobre toda la fórmula

Anatomía de una cláusula

$$\underbrace{A_0}_{\text{cabeza}} \leftarrow \underbrace{A_1 \wedge A_2 \wedge \dots \wedge A_n}_{\text{cuerpo}}$$

- Cada A_i es una declaración simple sin conectores
- A_0 es la **cabeza** de la cláusula
- $A_1 \wedge A_2 \wedge \dots \wedge A_n$ corresponde al **cuerpo**
- Si $n = 0$, se omite la implicancia y queda solo A_0
 - Esto indica que A_0 siempre es verdadero
 - Este tipo de cláusulas se denomina **hechos**

Prolog

- Hechos, reglas y consultas
- Operadores
- Variables y constantes
- El intérprete

Hechos

- Pueden escribirse de manera muy simple, como `juan` o bien `esta_lloviendo_ahora`
- Usualmente contienen predicados:

```
nino(pedro).  
nina(maria).  
amigos(pedro, maría).  
viajar(juan, valdivia).  
entregar(pedro, maría, lapiz).
```

- Los nombres de constantes y predicados comienzan con **minúscula**
- El predicado se escribe primero; los objetos que relaciona van entre paréntesis, separados por coma
- Cada hecho **siempre** termina con un punto (.)
- El orden de los argumentos es arbitrario, pero no da lo mismo: hay que ser **consistente**

Reglas

- Sirven para especificar dependencia entre hechos

```
hijo(Y, X)      :- padre(X, Y).
```

```
hermanos(X, Y) :- padre(Z, X), padre(Z, Y).
```

```
hermanos(X, Y) :- hermano(X, Y) ; hermano(Y, X).
```

Operadores lógicos

Operador	Significado
<code>:-</code>	Implicancia
<code>,</code>	AND
<code>;</code>	OR
<code>not</code>	Negación

Consultas / Preguntas

- Comienzan con ?- y terminan con .

```
?- nina(maria).
```

```
true.
```

Variables

- Sus nombres comienzan con letra **mayúscula**

```
?- nino(X).
```

```
X = pedro.
```

Constantes

- Números
- Palabras que comienzan con letra minúscula
- Palabras encerradas entre comillas

Unificación / Matching

- Es la manera en que Prolog calza dos términos
- Se tienen dos términos y se desea verificar si representan la misma estructura

Dado el hecho:

```
padre(juan, emilio).
```

y la consulta:

```
?- padre(X, Y).
```

```
X = juan, Y = emilio.
```

Unificación: interpretación

- Tal como esperábamos, X es instanciada a `juan` e Y a `emilio`
- Diremos entonces que:
 - el término `padre(X, Y)` es **unificado** con el término `padre(juan, emilio)`
 - con X ligado a `juan` e Y ligado a `emilio`

Unificación: más ejemplos

?- a = a.	% <i>true</i> .
?- a = b.	% <i>false</i> .
?- X = b.	% X = b.
?- ejemplo(p, q) = ejemplo(X, Y).	% X = p, Y = q.
?- ejemplo(p, q) = ejemplo(X, X).	% <i>false</i> .
?- [a,b,c] = [X Y].	% X = a, Y = [b,c].
?- [a,b,c] = [X,Y Z].	% X = a, Y = b, Z = [c].
?- [a,b,c] = [X,Y,Z T].	% X = a, Y = b, Z = c, T = [].

Algo de aritmética

- Algo natural en los paradigmas imperativo y funcional
- Algo especial debido a la naturaleza deductiva del paradigma lógico
- Revisaremos la diferencia entre igualdad aritmética y unificación:
 - = se usa para **unificación**
 - is indica **evaluar numéricamente** la expresión del lado derecho y unificarla con la del lado izquierdo
- Operadores: +, *, /, <, =<, >, >=

?- X is 1.	% X = 1.
?- A is 3+2.	% A = 5.
?- A is 2, B is A+3.	% A = 2, B = 5.
?- A is *(3, +(1,2)).	% A = 9.

Programando con listas

- Se definen con paréntesis cuadrados: `[1,2,3,4]`
- La lista vacía es `[]`
- Es posible usar patrones `[H | T]`, donde H es un elemento y T es una lista
- También es posible calzar patrones como `[1,2,3 | T]`

Encontrando el último elemento

- Se define un predicado que recibe una lista y su último elemento
 - El predicado será verdadero cuando efectivamente sea el último elemento

```
ultimo([H], H).
```

```
ultimo([_ | T], V) :- ultimo(T, V).
```

Consultas:

```
?- ultimo([1,2], 2).           % true.
```

```
?- ultimo([1,2,3], 2).        % false.
```

```
?- ultimo([1,2,3], X).        % X = 3.
```

```
?- ultimo([], X).             % false.
```

Sumando elementos

- Se define un predicado que recibe una lista y un número que representa el resultado de la suma
- La llamada es recursiva

```
sumar([], 0).
```

```
sumar([H | T], N) :- sumar(T, M), N is M + H.
```

```
?- sumar([1,2,3], X).           % X = 6.
```

La longitud de una lista

- Parecido al caso de la suma
- Se define un predicado que recibe una lista y un número que representa la cantidad de elementos
- La llamada también es recursiva, sumando 1 por cada sublista

```
largo([], 0).
```

```
largo([_ | T], N) :- largo(T, M), N is M + 1.
```

```
?- largo([1,2,3], X).           % X = 3.
```